DISEÑO DE COMPILADORES



Capítulo 2

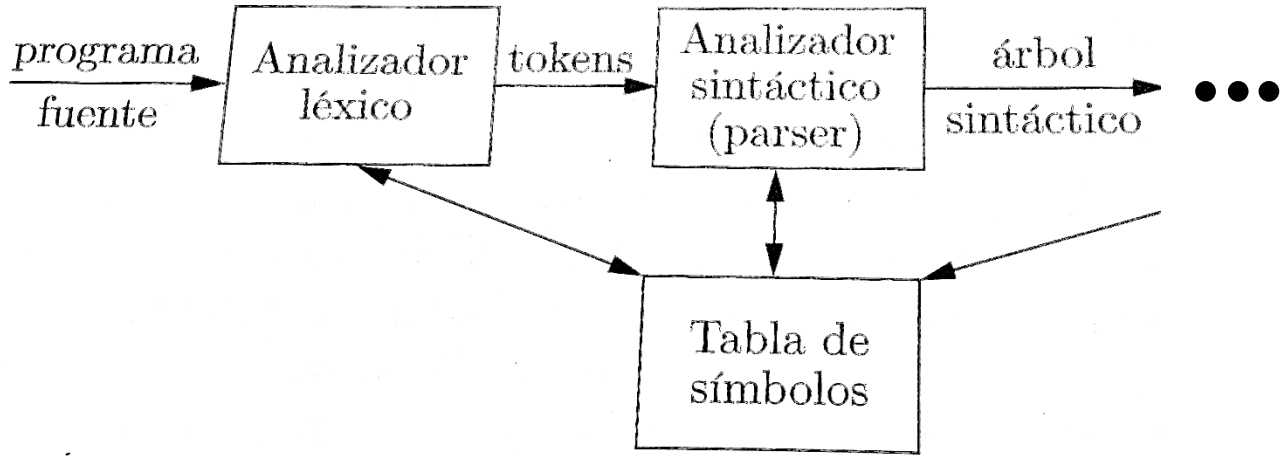
Un traductor simple orientado a la sintaxis

Observación. *El presente documento sirve de complemento al contenido que referencia y no se constituye en el detalle del mismo, ya que conceptos y contenidos pueden haber sido desarrollados en forma presencial, virtual o asignado como ejercicio. Ante cualquier duda se recomienda consultar con el profesor a cargo de la cátedra.*

Profesor: Sergio A. Aranda

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Fases de un compilador tradicional.



Notar que:

1. El *analizador léxico* genera e interactua con el *analizador sintáctico* mediante tokens.
2. Sólo el analizador léxico es el único que interactúa con el programa fuente en un compilador tradicional.
3. El *analizador sintáctico* interactúa con el *analizador semántico* a través del árbol sintáctico generado.

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Fases de un compilador simple o traductor dirigido por la sintáxis



Observaciones:

1. En un traductor dirigido por la sintáxis, el *analizador sintáctico* maneja el proceso.
2. El *analizador léxico* y *semántico* se subordinan al *sintáctico*, y actúan como procesos que se ejecutan dentro del mismo.
3. El *analizador sintáctico* interactúa con el lenguaje de entrada a diferencia de lo que ocurre en un compilador tradicional.

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Un analizador sintáctico cuenta con:

a. Una notación, que permite representar la sintáxis del **lenguaje de entrada** en forma de reglas sintácticas y la interacción entre ellas. La notación que se utiliza es la que se denomina BNF (*Backus Naur Form*).

b. Dos herramientas de validación que permiten aplicar las reglas sintácticas a cualquier lenguaje de entrada de manera a validar su construcción e identificando errores cuando corresponda.

   Las herramientas son:

   - *Árboles sintácticos*
   - *Derivaciones*

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Según *[Aho,2007]*, una gramática independiente de contexto en su representación BNF está formado por:

1. Un conjunto de *símbolos terminales*, que son los símbolos que aparecen explícitamente en una gramática o lenguaje de entrada.
2. Un conjunto de *no terminales* a los que se les conoce como variables sintácticas.
3. Un conjunto de *producciones* en forma jerárquica, formadas por:
    - *un lado izquierdo donde se encuentra un no terminal,*
    - *el símbolo → que se lee como ¨puede tener la forma de¨ o bien, ¨deriva en¨,*
    - *Un lado derecho formado por un conjunto de terminales y no terminales.*
4. Una designación de uno de los *no terminales* como *símbolo inicial*.

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Un árbol de análisis sintáctico muestra, en forma gráfica, la manera en que el símbolo inicial de una gramática deriva a una cadena en el lenguaje. Si el no terminal A tiene una producción $A \rightarrow XYZ$, entonces un árbol de análisis sintáctico podría tener un nodo interior etiquetado como A, con tres hijos llamados X, Y y Z, de izquierda a derecha:



De manera formal, dada una gramática libre de contexto, un *árbol de análisis sintáctico* de acuerdo con la gramática es un árbol con las siguientes propiedades:

1. La raíz se etiqueta con el símbolo inicial.
2. Cada hoja se etiqueta con un terminal, o con ϵ.
3. Cada nodo interior se etiqueta con un no terminal.
4. Si A es el no terminal que etiqueta a cierto nodo interior, y $X_1, X_2, \ldots, X_n$ son las etiquetas de los hijos de ese nodo de izquierda a derecha, entonces debe haber una producción $A \rightarrow X_1 X_2 \cdots X_n$. Aquí, cada una de las etiquetas $X_1, X_2, \ldots, X_n$ representa a un símbolo

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

cadena → *cadena* + *cadena*
cadena → *cadena* – *cadena*
cadena → *cadena*
cadena → **0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**

El BNF define un lenguaje de entrada formado por *suma* y *resta* de *digitos*.

Simbolos terminales: { 0, 1, 2, 3, 4, 5, 6, 7, 8 ,9, +, - }
Simbolos no terminales: { cadena }
La producción inicial es : cadena

Ejemplos de cadenas de entrada:
5 + 2 – 1 (es una cadena de entrada)
9 + 10 + 3 (no es una cadena de entrada)

EJERCICIO

Determinar cual de las siguientes gramáticas son ambigüas. Justificar obteniendo el árbol sintáctico correspondiente.

- $S \rightarrow 0\ S\ 1 \mid 0\ 1$
- $S \rightarrow S\ (\ S\ )\ S \mid \epsilon$
- $S \rightarrow a\ S\ b\ S \mid b\ S\ a\ S \mid \epsilon$

Para determinar si una gramática es ambigua, se debe obtener al menos dos árboles sintácticos para una cadena de entrada que pertenece al lenguaje

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Ejemplo: BNF para expresiones consistentes de operaciones de suma y resta de dígitos. Planteada en *[Aho, 2007]*

$$\textit{secuencia} \rightarrow \textit{secuencia} + \textit{digito}$$
$$\textit{secuencia} \rightarrow \textit{secuencia} - \textit{digito}$$
$$\textit{secuencia} \rightarrow \textit{digito}$$
$$\textit{digito} \rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9}$$

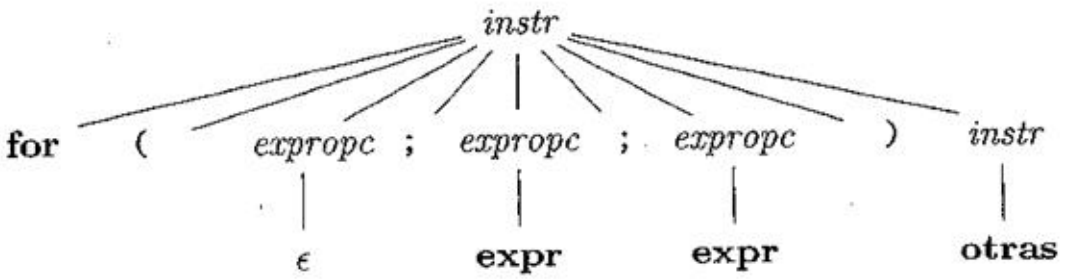
Terminales: { +, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
No Terminales: { *secuencia*, *digito*}
Símbolo inicial: *secuencia*

Validando mediante un árbol sintáctico con una estrategia de arriba-abajo

instr → **expr ;**
| **if (expr)** *instr*
| **for (** *expropc* **;** *expropc* **;** *expropc* **)** *instr*
| **otras**

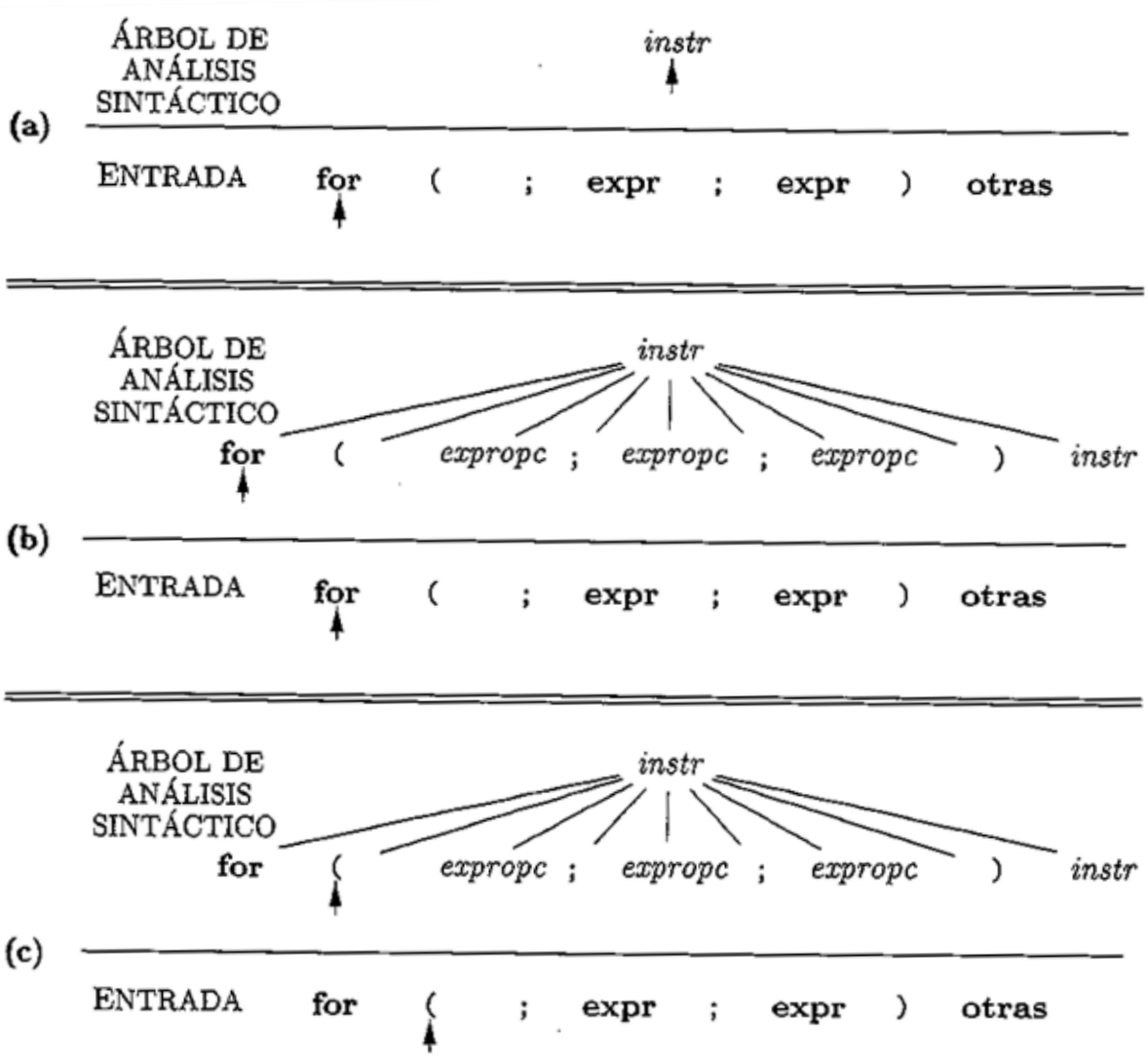
expropc → ϵ
| **expr**

Figura 2.16: Una gramática para algunas instrucciones en C y Java



ANALISIS SINTACTICO

Top-Down



TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS
Recursión por la izquierda

Eliminación de la recursión por la izquierda

Para toda producción que tenga la forma $A \rightarrow A\ \alpha \mid \beta$, *es posible reemplazarla por el siguiente sistema de producciones equivalente:*

$$A \rightarrow \beta\ R$$
$$R \rightarrow \alpha\ R \mid \varepsilon$$

Preguntas:

a. *¿Por qué es importante eliminar la recursión por la izquierda?*
b. *¿Que alternativas se plantea para producciones donde no se desee eliminar la recursión por la izquierda?*

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS

Definición del Conjunto PRIMERO

PRIMERO (X)

1. Si X es un terminal, entonces PRIMERO(X) = {X}
2. Si X es un no terminal y $X \rightarrow Y_1 Y_2 \ldots Y_k$ es una producción para $k \geq 1$, entonces se agrega **a** en PRIMERO(X) si para cierta i, **a** está en PRIMERO(Y_i), y ε está en todos las funciones PRIMERO(Y_1) … PRIMERO(Y_{i-1})
3. Si ε esta en todos los conjuntos PRIMERO(Y_1) a PRIMERO(Y_k), entonces se agrega ε en PRIMERO(X).
4. Si $X \rightarrow \varepsilon$ es una producción definida, entonces se agrega ε a PRIMERO(X).

PRIMERO ($X_1 X_2 \ldots X_n$)

1. Agregar a PRIMERO($X_1 X_2 \ldots X_n$) todos los símbolos que no sean ε de PRIMERO(X_1).
2. Si ε está en PRIMERO(X_1), agregar todos los símbolos que no sean ε de PRIMERO(X_1).
3. Si ε está en PRIMERO(X_1) y PRIMERO(X_2), entonces agregar todos los símbolos que no sean ε de ambos, y así sucesivamente.
4. Si ε está en todos los conjuntos PRIMERO de X_1 a X_n, entonces agregar ε al conjunto PRIMERO($X_1 X_2 \ldots X_n$)

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS
Conjunto PRIMERO

```
S -> A B | B A
A -> A a | Ɛ
B -> B b
----------

Conjunto PRIMERO
P(S)=P(A B) U P(B A)= {a,b,Ɛ}U{b,a,Ɛ}= P(a,b,Ɛ}

P(A)= P(A a)U P(Ɛ)= {a}U{Ɛ}= {a, Ɛ}

P(B)= esta indefinida, por lo tanto se recomienda eliminar la recursión
por la izq.
B -> R
R -> b R | Ɛ
P(B)= P(R)= {b, Ɛ}
P(R)= P(b B) U P(Ɛ) = {b, Ɛ}
```

TRADUCTOR SIMPLE ORIENTADO POR LA SINTAXIS
Gramática predictiva

Una gramática donde la entrada se procesa de izquierda a derecha un carácter a la vez y la recorrida del árbol sintáctico es por la izquierda (leftmost) se dice que es *predictiva*, cuando cumple las sgtes condiciones:

a. No tiene recursión por la izquierda, es decir, no contiene producciones de la forma $A \rightarrow A\ \alpha\ /\ \beta$,

b. No tiene ambigüedad sintáctica, es decir, para un no terminal con varias producciones, no existe *factor común* por la izquierda entre ellas.

c. No tiene ambigüedad semántica, es decir, un no terminal con varias producciones no deriva al mismo terminal por la izquierda. Esto se verifica obteniendo el *conjunto PRIMERO* de cada producción y verificando que sean disjuntos entre si.

ANALISIS SINTACTICO

Analizador Sintáctico Predictivo

```
instr() {
 switch ( preanálisis ) {
 case expr:
        coincidir(expr); coincidir(';'); break;
 case if:
        coincidir(if); coincidir('('); coincidir(expr); coincidir(')'); instr();
        break;
 case for;
        coincidir(for); coincidir('(');
        expropc(); coincidir(';'); expropc(); coincidir(';'); expropc();
        coincidir(')'); instr(); break;
 case otras;
        coincidir(otras); break;
 default:
        reportar("error de sintaxis");
 }


expropc() {
 if ( preanálisis == expr ) coincidir(expr);


coincidir(terminal t) {
 if ( preanálisis == t ) preanálisis = siguienteTerminal;
 else reportar("error de sintaxis");
```

TRADUCCIÓN DIRIGIDA POR LA SINTAXIS
REGLAS SEMÁNTICAS

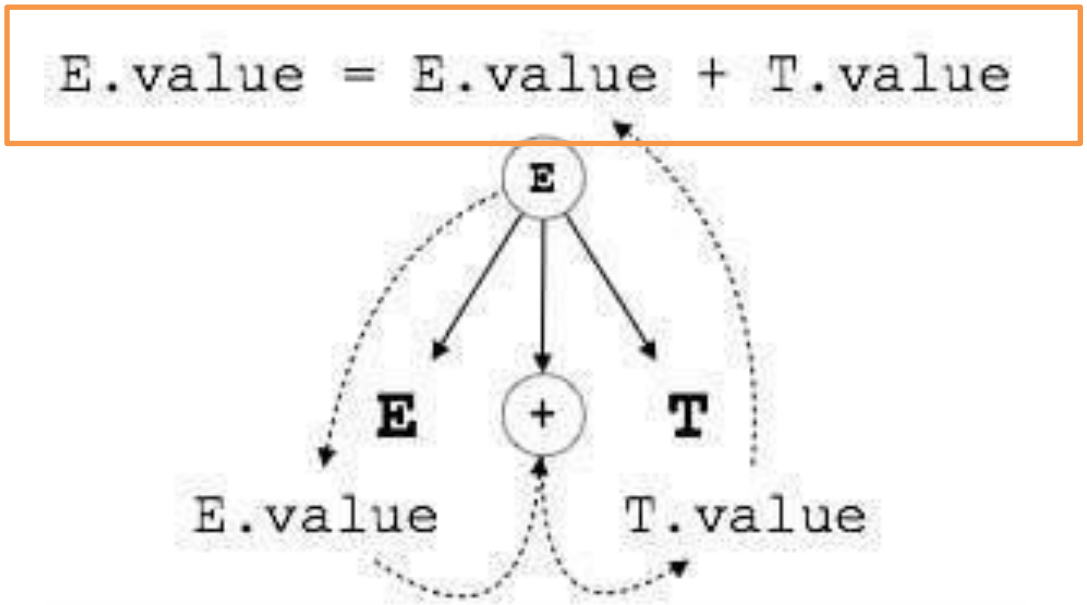
Se realiza *uniendo* reglas o fragmentos de un programa a las producciones de una gramática.

ATRIBUTOS.
Cada no terminal es una variable sintáctica, que puede tener asociada atributos o instancias de variables que se utilizan para guardar valores. Existen dos tipos de estos atributos:

a. Sintetizados, son aquellos cuyo valor se obtiene por la evaluación de sus hijos en un árbol sintáctico.

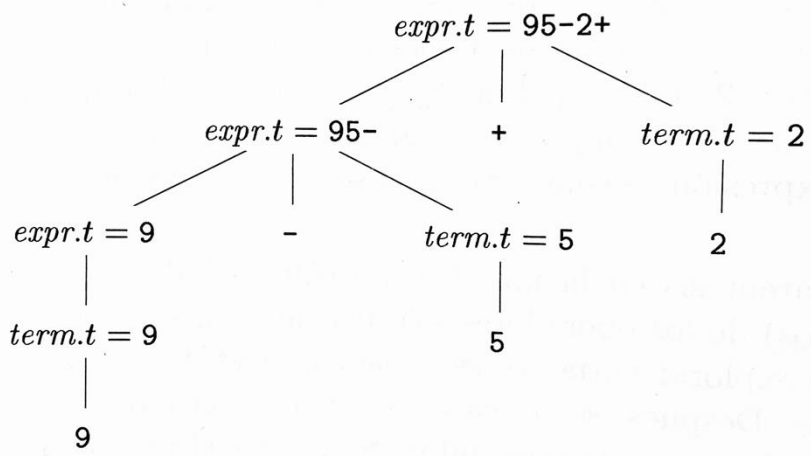
a. Heredados, son aquellos cuyo valor se puede obtener a partir del padre o de sus hermanos en un árbol sintáctico.

E -> E + T



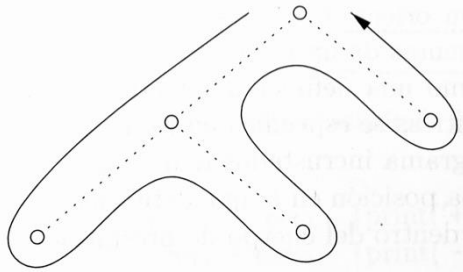
TRADUCCIÓN DIRIGIDA POR LA SINTAXIS
REGLAS SEMÁNTICAS

Ejemplo: Convertir cadenas de sumas y restas en formato infijo a forma posfija.




```
procedure visitar(nodo N) {
    for ( cada hijo (C de N, de izquierda a derecha ) {
        visitar(C);
    }
    evaluar las reglas semánticas en el nodo N;
}
```

Figura 2.11: Un recorrido tipo "primero en profundidad" de un árbol



TRADUCCIÓN DIRIGIDA POR LA SINTAXIS
REGLAS SEMÁNTICAS

Ejemplo: Convertir cadenas de sumas y restas en formato infijo a forma posfija.

PRODUCCIÓN	REGLAS SEMÁNTICAS
$expr \rightarrow expr_1 + term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '+'$
$expr \rightarrow expr_1 - term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '-'$
$expr \rightarrow term$	$expr.t = term.t$
$term \rightarrow 0$	$term.t = '0'$
$term \rightarrow 1$	$term.t = '1'$
...	...
$term \rightarrow 9$	$term.t = '9'$

TRADUCCIÓN DIRIGIDA POR LA SINTAXIS
ESQUEMAS DE TRADUCCIÓN

Insertar pedazos de programa que realiza la traducción dentro del BNF. El órden de evaluación de las reglas semánticas queda especificado explícitamente.

REGLAS SEMÁNTICAS

PRODUCCIÓN	REGLAS SEMÁNTICAS
$expr \rightarrow expr_1 + term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '+'$
$expr \rightarrow expr_1 - term$	$expr.t = expr_1.t \mid\mid term.t \mid\mid '-'$
$expr \rightarrow term$	$expr.t = term.t$
$term \rightarrow 0$	$term.t = '0'$
$term \rightarrow 1$	$term.t = '1'$
...	...
$term \rightarrow 9$	$term.t = '9'$

ESQUEMA DE TRADUCCIÓN

$$
\begin{array}{lcll}
expr & \rightarrow & expr_1 + term & \{\text{print}('+')\} \\
expr & \rightarrow & expr_1 - term & \{\text{print}('-')\} \\
expr & \rightarrow & term & \\
term & \rightarrow & 0 & \{\text{print}('0')\} \\
term & \rightarrow & 1 & \{\text{print}('1')\} \\
 & & \ldots & \\
term & \rightarrow & 9 & \{\text{print}('9')\}
\end{array}
$$



Esquemas de traducción vistas en el árbol sintáctico, con el órden de evaluación

